

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



LÊ NHƯ ĐẠT - 52200160

**BÁO CÁO GIỮA KỲ
ĐẠI SỐ TUYẾN TÍNH ỨNG DỤNG CHO
CÔNG NGHỆ THÔNG TIN**

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2024

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



LÊ NHƯ ĐẠT - 52200160

**BÁO CÁO GIỮA KỲ
ĐẠI SỐ TUYẾN TÍNH ỨNG DỤNG CHO
CÔNG NGHỆ THÔNG TIN**

Người hướng dẫn
ThS. Phạm Quốc Duy

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2024

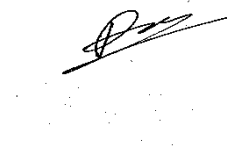
LỜI CẢM ƠN

Chúng em xin chân thành đến thầy Duy vì đã tận tình hướng dẫn, truyền đạt kiến thức và dành thời gian quý báu hỗ trợ chúng em trong suốt quá trình thực hiện bài báo cáo này. Những góp ý và chỉ dẫn của thầy đã giúp chúng em hiểu sâu hơn về đề tài, hoàn thiện kỹ năng nghiên cứu và trình bày. Chúng em xin trân trọng cảm ơn thầy và kính chúc thầy luôn mạnh khỏe, thành công trong sự nghiệp trồng người.

TP. Hồ Chí Minh, ngày 13 tháng 12 năm 2024

Tác giả

(Ký tên và ghi rõ họ tên)



Lê Như Đạt

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của ThS Phạm Quốc Duy. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 13 tháng 12 năm 2024

Tác giả

(Ký tên và ghi rõ họ tên)




Lê Như Đạt

TÓM TẮT

Bài báo cáo được hoàn tất 100% với các những vấn đề cần được giải quyết là Programming part và Report part. Giải quyết các vấn đề trong bài báo cáo bằng những tư duy logic và những gì đã học trong môn Đại số tuyến tính ứng dụng cho công nghệ thông tin.

MỤC LỤC

DANH MỤC HÌNH VẼ	vii
CHƯƠNG 1. PHƯƠNG PHÁP GIẢI QUYẾT NHIỆM VỤ	1
1.1 Mô tả ngắn về chương trình.....	1
1.2 Giải thích chi tiết các hàm	3
1.2.1 Chuẩn bị dữ liệu.....	3
1.2.2 Hàm Ex_a()	4
1.2.3 Hàm Ex_b().....	5
1.2.4 Hàm Ex_c()	6
1.2.5 Hàm isPrime	9
1.2.6 Hàm Ex_d().....	9
1.2.7 Hàm getMaxRowIndices(arr) -> list.....	10
1.2.8 Hàm Ex_e()	11
1.2.9 Hàm Ex_f()	12
CHƯƠNG 2. KẾT QUẢ	13
2.1 Lần chạy đầu tiên.....	13
2.2 Lần chạy thứ 2	16
2.3 Lần chạy thứ 3	19
2.4 Lần chạy thứ 4	22
2.5 Lần chạy thứ 5	26
TÀI LIỆU THAM KHẢO	31

DANH MỤC HÌNH VẼ

Hình 1. Chuẩn bị dữ liệu mẫu	2
Hình 2. Không sử dụng phương pháp vectorize	2
Hình 3. Phương pháp vectorize.....	3
Hình 4. Chuẩn bị dữ liệu	4
Hình 5. Hàm Ex_a.....	5
Hình 6. Hàm Ex_b.....	6
Hình 7. Hàm Ex_c.....	7
Hình 8. Ví dụ Ex_c (1).....	8
Hình 9. Ví dụ Ex_c (2).....	8
Hình 10. Kết quả ví dụ Ex_c.....	9
Hình 12. Hàm Ex_d.....	10
Hình 13. Hàm getMaxRowIndices.....	10
Hình 14. Hàm Ex_e.....	11
Hình 15. Hàm Ex_f	12
Hình 16. 1 st input.....	14
Hình 17. 1 st output.....	16
Hình 18. 2 nd input.....	16
Hình 19. 2 nd output.....	19
Hình 20. 3 rd input	20
Hình 21. 3 rd output	22
Hình 22. 4 th input.....	23
Hình 23. 4 th output.....	26
Hình 24. 5 th input.....	27
Hình 25. 5 th output.....	30

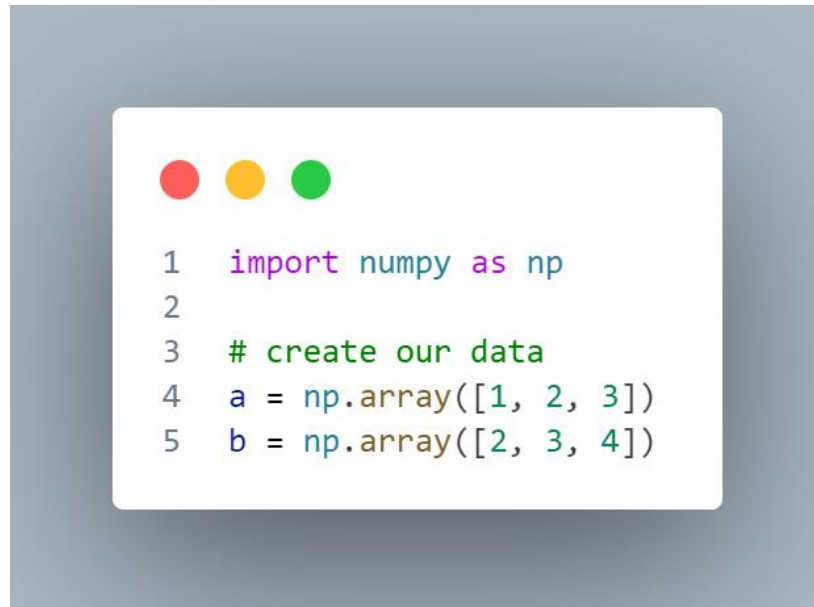
CHƯƠNG 1. PHƯƠNG PHÁP GIẢI QUYẾT NHIỆM VỤ

1.1 Mô tả ngắn về chương trình

Chương trình dùng NumPy để xử lý ma trận, kết hợp với các hàm số học như math và vectorize. Mỗi hàm con trong chương trình thực hiện một nội dung riêng lẻ:

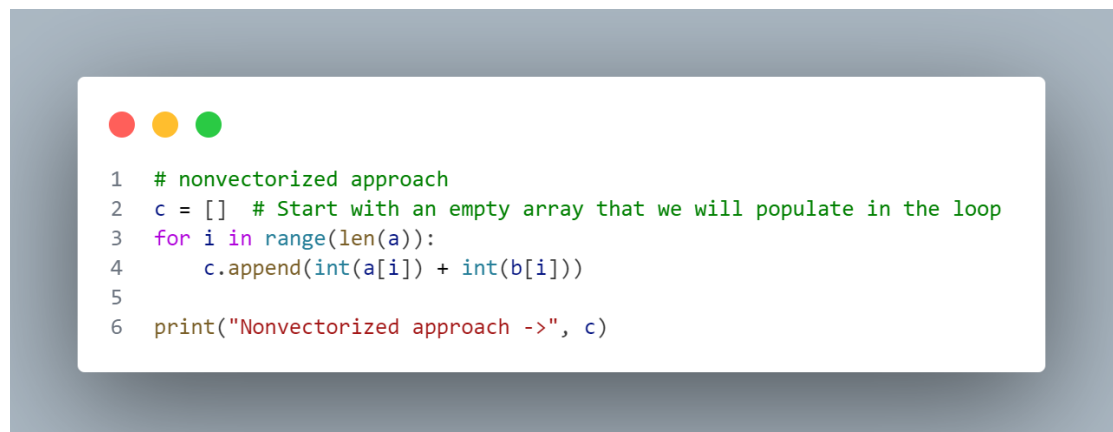
- **Ex_a:** Tính tổng ma trận sau khi kết hợp các biến đổi ma trận.
- **Ex_b:** Tính tổng dạng ma trận vào lớp lũy thừa dựa trên dãy chia liên tiếp.
- **Ex_c:** Tạo vector từ các phần tử lẻ trong ma trận.
- **Ex_d:** Lọc các số nguyên tố trong ma trận.
- **Ex_e:** Tìm dòng chứa nhiều số nguyên tố nhất.
- **Ex_f:** Tìm dòng chứa nhiều cặp số lẻ liên tiếp nhất.

Vậy ‘vectorize’ là gì? Vectorizing code là một kỹ thuật thường cho phép bạn tạo mã nhanh hơn và dễ đọc hơn. Vector hóa là quá trình thực hiện tính toán trên một tập hợp các giá trị cùng một lúc thay vì lặp một cách rõ ràng qua từng phần tử riêng lẻ. Sự khác biệt có thể dễ dàng thấy được trong một ví dụ đơn giản. Giả sử chúng ta muốn cộng từng phần tử của hai mảng lại với nhau, $a = [a_0, a_1, a_2]$ và $b = [b_0, b_1, b_2]$ để chúng ta tạo ra một mảng mới $c = [a_0 + b_0, a_1 + b_1, a_2 + b_2]$. Chúng ta hãy cùng xem cách thực hiện điều này theo cách không vector hóa (sử dụng vòng lặp) và sử dụng vector hóa.



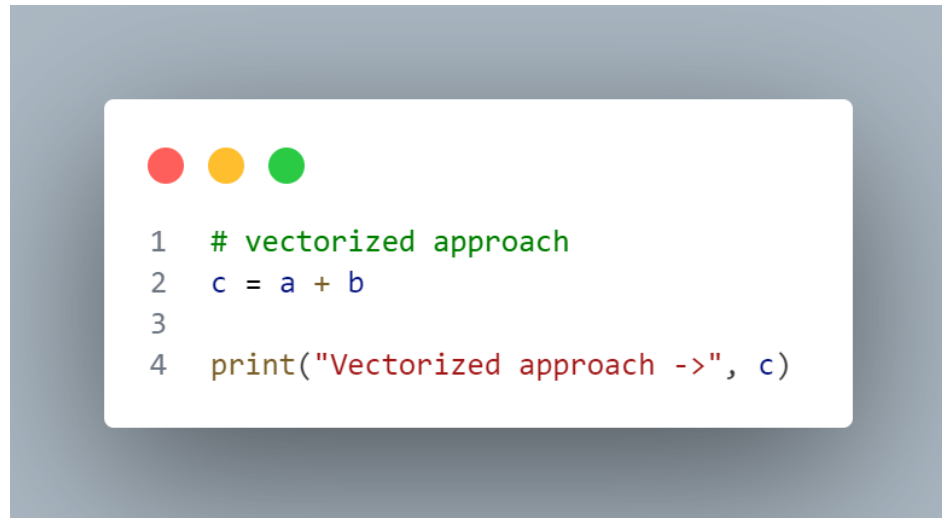
Hình 1. Chuẩn bị dữ liệu mẫu

Trước tiên, chúng ta hãy hoàn thành việc này bằng cách sử dụng phương pháp không vector hóa sử dụng vòng lặp for...



Hình 2. Không sử dụng phương pháp vectorize

Bây giờ, chúng ta hãy thử phương pháp vector hóa...



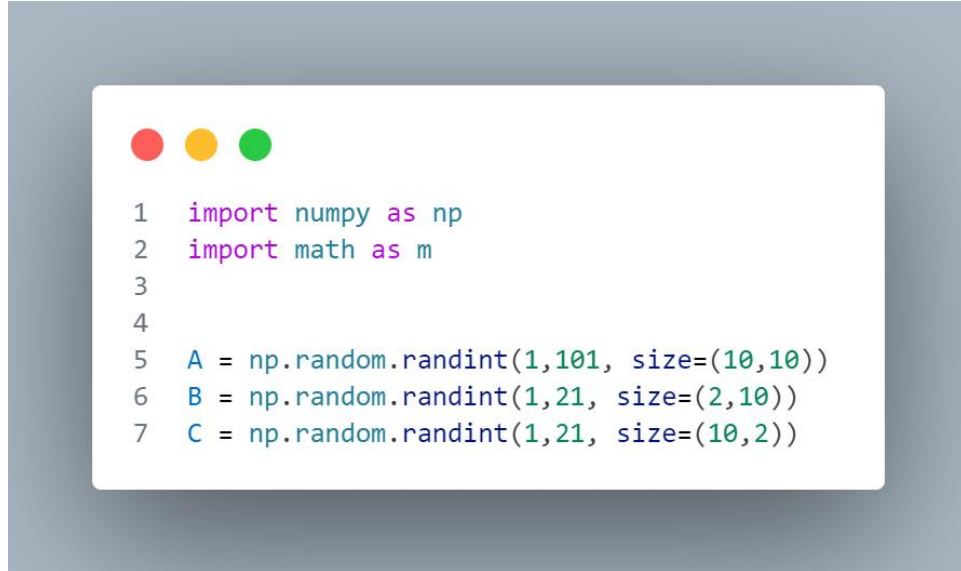
Hình 3. Phương pháp vectorize

Một điều rõ ràng ngay lập tức - ký hiệu của phương pháp vector hóa rõ ràng hơn nhiều so với phương pháp không vector hóa. Nó nói chính xác những gì đang xảy ra - một phép cộng vector theo từng phần tử. Phương pháp này thậm chí còn rõ ràng hơn với ma trận 2D vì ma trận 2D sẽ yêu cầu các vòng lặp lồng nhau. Trong khoa học dữ liệu, chúng ta thường làm việc với các vector hoặc ma trận dữ liệu và cần thực hiện các phép toán theo từng phần tử trên chúng như chúng ta đã thảo luận trước đó, vì vậy ký hiệu vector hóa thường được ưu tiên hơn để làm rõ mã.

Trong phần “Programming Part” phần bài tập được áp dụng phương pháp này chính là **Ex_d**.

1.2 Giải thích chi tiết các hàm

1.2.1 Chuẩn bị dữ liệu



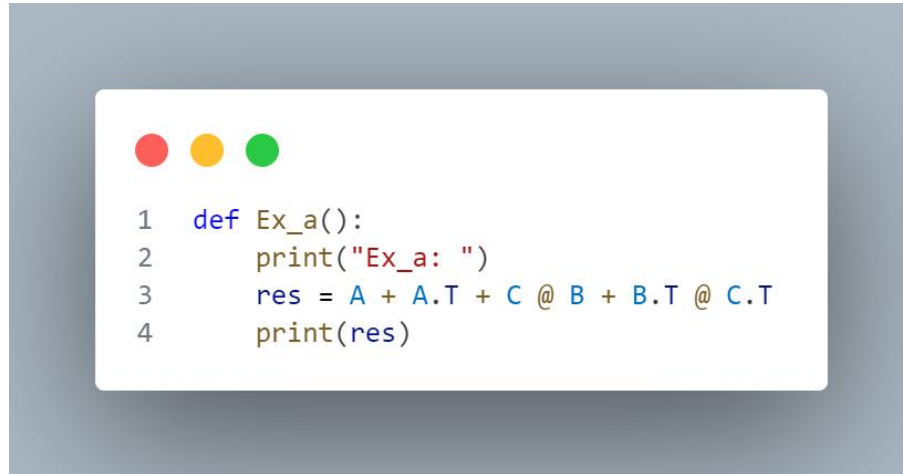
Hình 4. Chuẩn bị dữ liệu

Trong phần bài tập này, ta sẽ thêm hai thư viện giúp hỗ trợ trong việc giải bài tập trong phần “Programing Part” đó chính là thư viện Numpy và thư viện Math.

Numpy (Numeric Python): là một thư viện toán học phổ biến và mạnh mẽ của Python. Cho phép làm việc hiệu quả với ma trận và mảng, đặc biệt là dữ liệu ma trận và mảng lớn với tốc độ xử lý nhanh hơn nhiều lần khi chỉ sử dụng “core Python” đơn thuần.

- A: Tạo ma trận ngẫu nhiên 10x10 với giá trị từ 1 đến 100.
- B: Tạo ma trận ngẫu nhiên 2x10 với giá trị từ 1 đến 20.
- C: Tạo ma trận ngẫu nhiên 10x2 với giá trị từ 1 đến 20.

1.2.2 Hàm *Ex_a()*



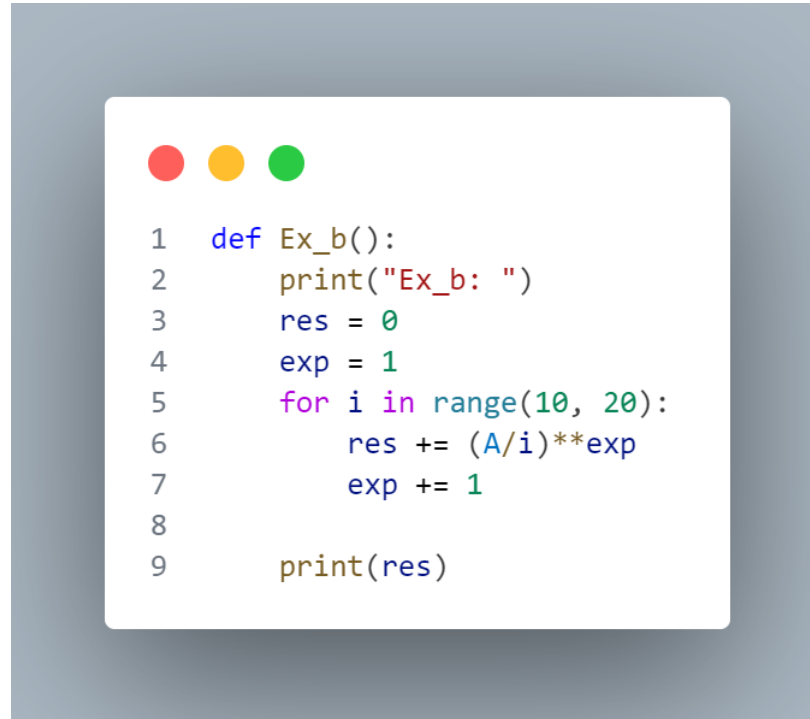
Hình 5. Hàm Ex_a

Hàm này tính tổng kết hợp các biến đổi ma trận được cho bởi công thức sau:

$$A + A^T + CB + B^T C^T$$

- A.T: Ma trận chuyển vị của A.
- C @ B: Tích ma trận giữa C và B.
- B.T @ C.T: Tích ma trận giữa ma trận chuyển vị của B và C.
- res: Ma trận kết quả là tổng các ma trận.

1.2.3 Hàm Ex_b()



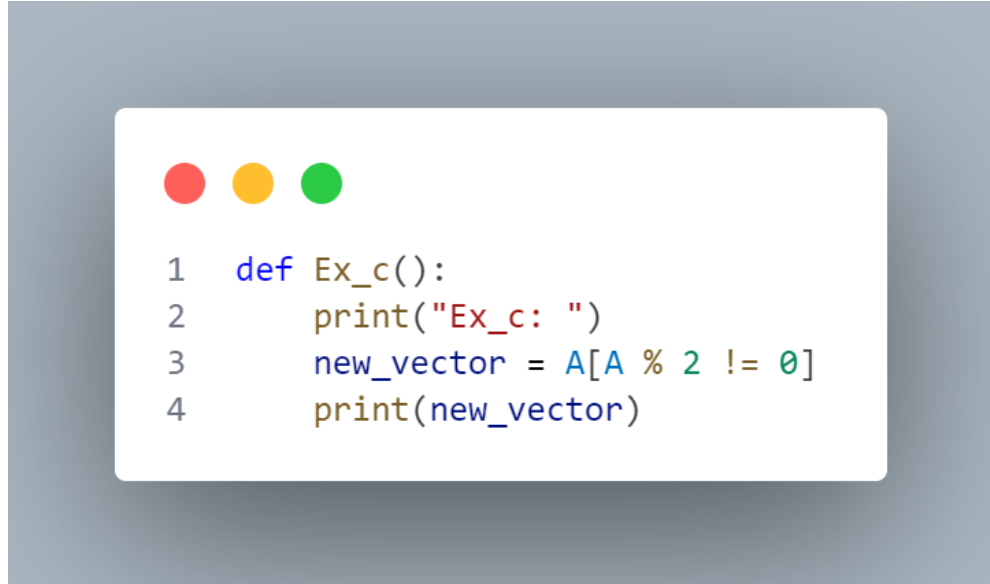
Hình 6. Hàm Ex_b

Hàm này tính tổng lũy thừa chia theo chỉ số trong khoảng từ 10 đến 19 theo công thức sau:

$$\frac{A}{10} + \left(\frac{A}{11}\right)^2 + \left(\frac{A}{12}\right)^3 + \dots + \left(\frac{A}{17}\right)^8 + \left(\frac{A}{18}\right)^9 + \left(\frac{A}{19}\right)^{10}$$

- res: Biến tích lũy kết quả tính tổng.
- (A / i): Chia từng phần tử của ma trận A cho số i.
- **exp: Lũy thừa theo chỉ số exp.
- exp += 1: Tăng lũy thừa sau mỗi lần lặp.

1.2.4 Hàm Ex_c()



Hình 7. Hàm Ex_c

Hàm **Ex_c** sử dụng kỹ thuật **boolean indexing** của NumPy để lọc và trích xuất các phần tử thỏa mãn điều kiện các phần tử là số lẻ trong ma trận A.

- **A % 2 != 0:**
 - Kiểm tra từng phần tử của ma trận A để xác định phần tử nào là số lẻ.
 - Kết quả trả về là một ma trận Boolean cùng kích thước với A, chứa True tại các vị trí số lẻ và False tại các vị trí còn lại.
- **A[A % 2 != 0]:**
 - Đây là **boolean indexing**: Chọn ra tất cả các phần tử trong ma trận A tại các vị trí mà giá trị Boolean tương ứng là True.
 - Kết quả trả về là một mảng 1D chứa toàn bộ các số lẻ trong ma trận A.
- **new_vector:**
 - Lưu trữ mảng 1D kết quả chứa các số lẻ.
- **In kết quả:**
 - `print(new_vector)` hiển thị mảng các số lẻ.

Ví dụ:

Giả sử ma trận A là:



```
1 A = np.array([[ 1,  2,  3],
2               [ 4,  5,  6],
3               [ 7,  8,  9]])
```

Hình 8. Ví dụ Ex_c (1)

A % 2 != 0 trả về:



```
1 [[ True, False,  True],
2  [False,  True, False],
3  [ True, False,  True]]
```

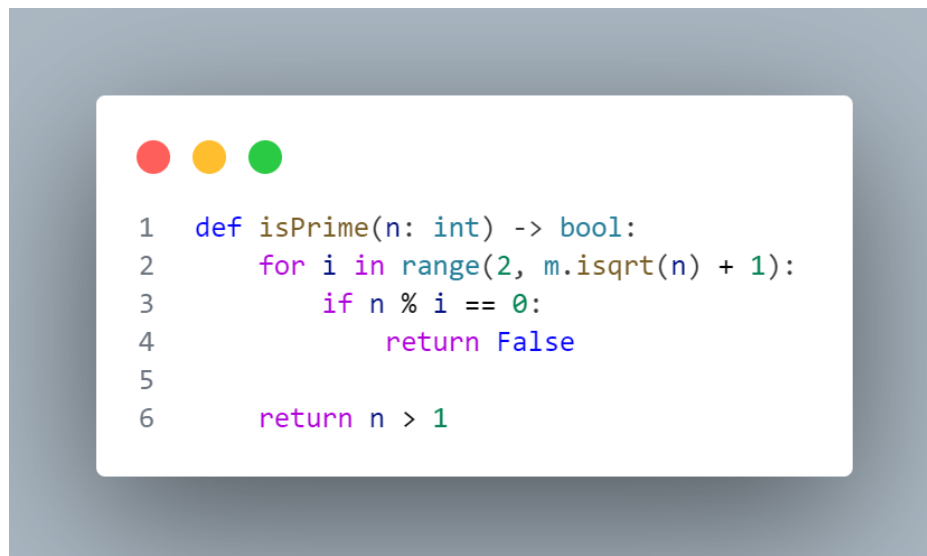
Hình 9. Ví dụ Ex_c (2)

A[A % 2 != 0] trả về:



Hình 10. Kết quả ví dụ Ex_c

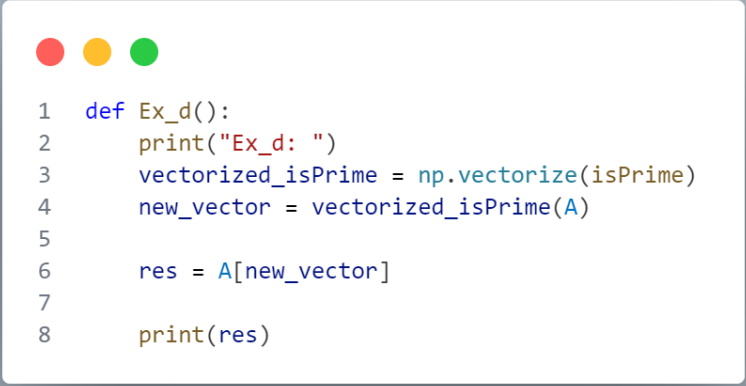
1.2.5 Hàm *isPrime*



Hình 11. Hàm isPrime

Hàm này nhận vào một số nguyên n và trả về True nếu n là số Nguyên tố, và ngược lại.

1.2.6 Hàm *Ex_d()*



```

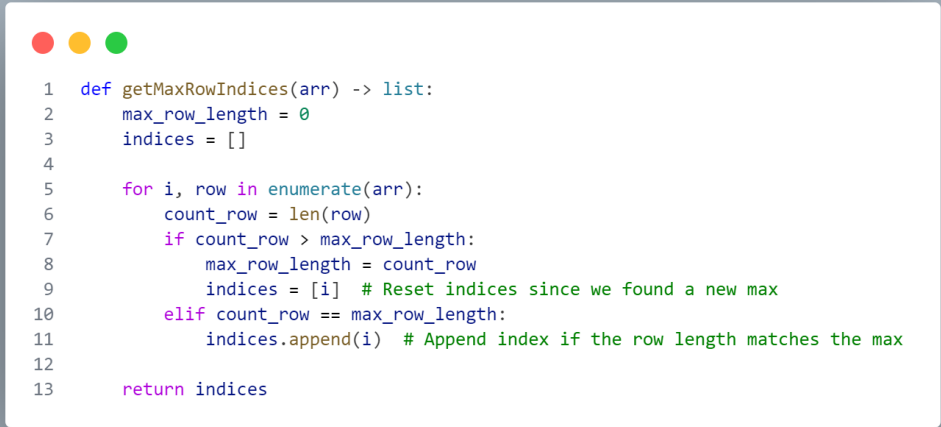
1  def Ex_d():
2      print("Ex_d: ")
3      vectorized_isPrime = np.vectorize(isPrime)
4      new_vector = vectorized_isPrime(A)
5
6      res = A[new_vector]
7
8      print(res)

```

Hình 12. Hàm Ex_d

- `np.vectorize(isPrime)`: Biến hàm `isPrime` thành `vectorized` để áp dụng cho ma trận.
- `A[new_vector]`: Trả về các phần tử nguyên tố trong `A`

1.2.7 Hàm *getMaxRowIndices(arr) -> list*



```

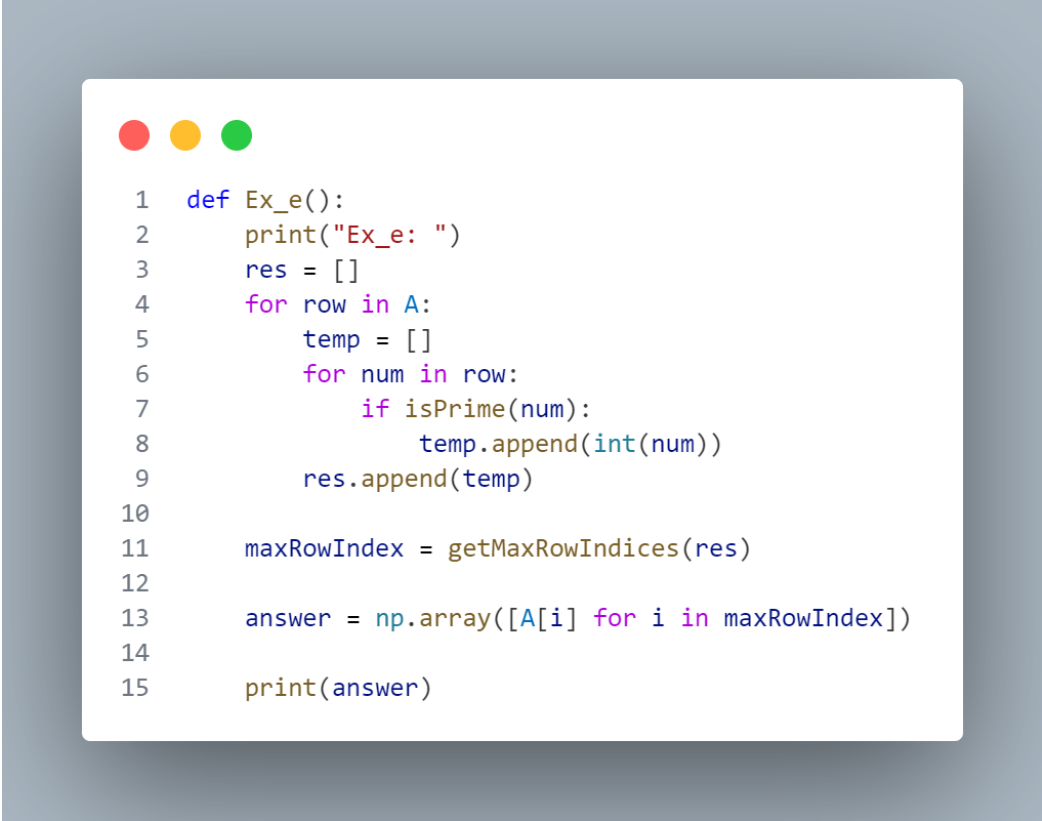
1  def getMaxRowIndices(arr) -> list:
2      max_row_length = 0
3      indices = []
4
5      for i, row in enumerate(arr):
6          count_row = len(row)
7          if count_row > max_row_length:
8              max_row_length = count_row
9              indices = [i] # Reset indices since we found a new max
10         elif count_row == max_row_length:
11             indices.append(i) # Append index if the row length matches the max
12
13     return indices

```

Hình 13. Hàm getMaxRowIndices

Hàm này nhận vào một mảng hai chiều và trả về một list chứa các chỉ số dòng có nhiều phần tử nhất. Ban đầu khởi tạo hai biến `max_row_length` và `indices` lần lượt có các giá trị là 0 và empty list. Sau đó duyệt qua các dòng trong mảng và lấy ra dòng có nhiều phần tử nhất. Đây là hàm giúp hiện thực câu e và f dễ dàng hơn.

1.2.8 Hàm `Ex_e()`



```

1  def Ex_e():
2      print("Ex_e: ")
3      res = []
4      for row in A:
5          temp = []
6          for num in row:
7              if isPrime(num):
8                  temp.append(int(num))
9          res.append(temp)
10
11     maxRowIndex = getMaxRowIndices(res)
12
13     answer = np.array([A[i] for i in maxRowIndex])
14
15     print(answer)

```

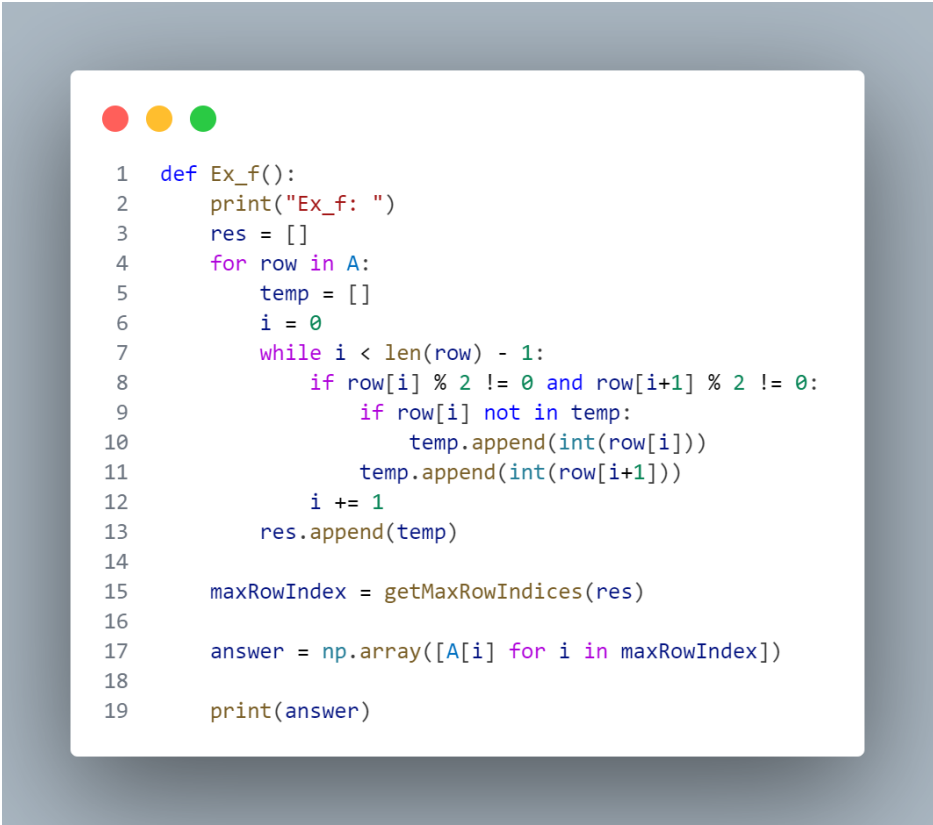
Hình 14. Hàm `Ex_e`

Hàm này tìm những dòng có nhiều số nguyên tố nhất, cụ thể như sau. Ban đầu tạo biến `res` và gán giá trị empty list. Sau đó lặp qua từng dòng trong ma trận `A`, tạo một biến `temp` cũng có giá trị empty list. Tiếp tục ta duyệt qua từng phần tử trong `row` và kiểm tra xem nếu phần tử đó là số nguyên tố thì ta thêm vào `temp`. Khi kết thúc vòng lặp con thì thêm `temp` vào `res`. Hàm `getMaxRowIndices` sẽ trả về list các indices có indices

kết quả. Cuối cùng ta chỉ cần lấy những vị trí đã tìm được lưu vào answer thì sẽ có đáp án.

- temp.append: Lưu các phần tử nguyên tố vào danh sách tạm thời.
- maxRowIndex: Tìm chỉ số các dòng chứa nhiều nguyên tố nhất.
- answer: Trả về ma trận có các dòng tìm được.

1.2.9 Hàm Ex_f()



```

1  def Ex_f():
2      print("Ex_f: ")
3      res = []
4      for row in A:
5          temp = []
6          i = 0
7          while i < len(row) - 1:
8              if row[i] % 2 != 0 and row[i+1] % 2 != 0:
9                  if row[i] not in temp:
10                     temp.append(int(row[i]))
11                     temp.append(int(row[i+1]))
12                 i += 1
13             res.append(temp)
14
15     maxRowIndex = getMaxRowIndex(res)
16
17     answer = np.array([A[i] for i in maxRowIndex])
18
19     print(answer)

```

Hình 15. Hàm Ex_f

Hàm này tìm những dòng có nhiều chuỗi những số lẻ liên tiếp, cụ thể như sau. Ban đầu tạo biến res và gán giá trị empty list. Sau đó lặp qua từng dòng trong ma trận A, tạo một biến temp cũng có giá trị empty list. Tiếp tục ta duyệt qua từng phần tử trong row và kiểm tra xem nếu phần tử đó và phần tử kế tiếp đều là số lẻ và nếu như phần tử

lẻ đó chưa được thêm vào temp thì ta thêm vào temp. Khi kết thúc vòng lặp while thì thêm temp vào res. Hàm getMaxRowIndices sẽ trả về list các indices có indices kết quả. Cuối cùng ta chỉ cần lấy những vị trí đã tìm được lưu vào answer thì sẽ có đáp án.

- while $i < \text{len}(\text{row}) - 1$: Lần lượt duyệt qua từng cặp phần tử trong dòng.
- $\text{row}[i] \% 2 \neq 0$: Kiểm tra phần tử chẵn lẻ.
- temp.append : Lưu các phần tử lẻ liên tiếp vào danh sách tạm thời.
- maxRowIndex : Xác định dòng chứa nhiều cặp số lẻ liên tiếp nhất.
- answer : Ma trận chứa các dòng đó.

CHƯƠNG 2. KẾT QUẢ

2.1 Lần chạy thứ 1

```

Matrix A:
[[ 43  78  48  73  34  24  80  86  23  91]
 [ 44  33  12  20  37   1   3  36  80  23]
 [ 19  54  92  51 100  65  68  54  61   1]
 [ 74 100  29  45  82   4  95  87  78  44]
 [ 60  46  76  25  76  30  77  44  65  11]
 [ 66  35  36  19   7  74  13  54  12  32]
 [ 80  66  73  81  24  58   7  99  70   8]
 [ 80  51  30   4  55  76  28   9  19  17]
 [ 42   2  33  78  35  57  53  10   1  92]
 [ 95  43  31  64  78  51  12  37  76  31]]

Matrix B:
[[ 6 20 14 11  2 14  6 10 16 17]
 [ 3  3  8 16 13 17 14 10  7  4]]

Matrix C:
[[14  9]
 [ 5 11]
 [ 1 17]
 [19  3]
 [15  4]
 [ 1  8]
 [10  3]
 [ 6 11]
 [ 6 18]
 [20 10]]

```

Hình 16. 1st input

```

Ex_a:
[[308 492 392 568 341 469 439 465 442 610]
 [492 332 295 740 548 337 462 400 413 625]
 [392 295 484 653 641 482 549 436 457 477]
 [568 740 653 604 413 479 490 553 835 823]
 [341 548 641 413 316 421 306 444 614 530]
 [469 337 482 479 421 448 380 491 531 582]
 [439 462 549 490 306 380 218 447 592 462]
 [465 400 436 553 444 491 447 358 442 500]
 [442 413 457 835 614 531 592 442 446 732]
 [610 625 477 823 530 582 462 500 732 822]]

```

Ex_b:

```
[ [9.76040682e+03 2.18697607e+06 2.54563046e+04 1.17292656e+06
  1.39705727e+03 1.14884462e+02 2.77748331e+06 5.51196186e+06
  8.82218690e+01 9.43987454e+06]
[1.19000372e+04 1.10492833e+03 5.07215057e+00 3.97955983e+01
  2.76078712e+03 1.08880130e-01 3.93366218e-01 2.20832794e+03
  2.77748331e+06 8.82218690e+01]
[3.05412197e+01 7.28720782e+04 1.04774306e+07 4.36142981e+04
  2.32683564e+07 3.97543528e+05 6.04449717e+05 7.28720782e+04
  2.21239280e+05 1.08880130e-01]
[1.33262411e+06 2.32683564e+07 4.16813571e+02 1.44606647e+04
  3.50830858e+06 5.80567199e-01 1.42364795e+07 6.15224538e+06
  2.18697607e+06 1.19000372e+04]
[1.90090548e+05 1.75154217e+04 1.71225209e+06 1.49372962e+02
  1.71225209e+06 5.34672010e+02 1.93651777e+06 1.19000372e+04
  3.97543528e+05 3.95729622e+00]
[4.57991148e+05 1.75978154e+03 2.20832794e+03 3.05412197e+01
  1.43324217e+00 1.33262411e+06 6.50886125e+00 7.28720782e+04
  5.07215057e+00 8.70628432e+02]
[2.77748331e+06 4.57991148e+05 1.17292656e+06 3.12365659e+06
  1.14884462e+02 1.39400192e+05 1.43324217e+00 2.11305399e+07
  7.92024810e+05 1.86364462e+00]
[2.77748331e+06 4.36142981e+04 5.34672010e+02 5.80567199e-01
  8.60188419e+04 1.71225209e+06 3.23874665e+02 2.40397444e+00
  3.05412197e+01 1.80702704e+01]
[7.97838227e+03 2.38313319e-01 1.10492833e+03 2.18697607e+06
  1.75978154e+03 1.18964327e+05 6.15757328e+04 3.08726437e+00
  1.08880130e-01 1.04774306e+07]
[1.42364795e+07 9.76040682e+03 6.83497590e+02 3.44406121e+05
  2.18697607e+06 4.36142981e+04 5.07215057e+00 2.76078712e+03
  1.71225209e+06 6.83497590e+02]]
```

```

Ex_c:
[43 73 23 91 33 37 1 3 23 19 51 65 61 1 29 45 95 87 25 77 65 11 35 19
 7 13 73 81 7 99 51 55 9 19 17 33 35 57 53 1 95 43 31 51 37 31]
Ex_d:
[43 73 23 37 3 23 19 61 29 11 19 7 13 73 7 19 17 2 53 43 31 37 31]
Ex_e:
[[95 43 31 64 78 51 12 37 76 31]]
Ex_f:
[[ 74 100 29 45 82 4 95 87 78 44]
 [ 80 66 73 81 24 58 7 99 70 8]]

```

Hình 17. 1st output

2.2 Lần chạy thứ 2

```

Matrix A:
[[ 40 47 78 71 58 54 48 40 79 72]
 [ 11 47 53 18 22 38 91 4 73 88]
 [ 68 87 71 95 3 90 9 96 82 73]
 [ 77 51 3 93 28 40 49 45 81 55]
 [ 95 46 80 57 88 54 8 55 94 45]
 [100 18 60 85 4 38 62 4 80 12]
 [ 86 87 79 92 38 56 1 53 57 64]
 [ 67 76 1 9 21 64 54 2 42 57]
 [ 79 74 79 38 66 25 10 59 61 60]
 [ 78 38 28 74 68 18 84 6 25 70]]
Matrix B:
[[ 1 4 13 3 5 14 19 10 8 10]
 [12 9 18 9 4 19 15 17 6 4]]
Matrix C:
[[14 9]
 [17 13]
 [16 16]
 [17 15]
 [11 6]
 [16 12]
 [20 3]
 [14 16]
 [ 1 18]
 [ 1 17]]

```

Hình 18. 2nd input

```
Ex_a:  
[[ 324  368  698  468  342  681  591  606  541  531]  
 [ 368  464  803  440  303  713  803  671  527  505]  
 [ 698  803 1134  781  478 1102  946  999  722  644]  
 [ 468  440  781  558  317  804  776  665  510  515]  
 [ 342  303  478  317  334  454  457  422  361  320]  
 [ 681  713 1102  804  454  980  939  932  661  575]  
 [ 591  803  946  776  457  939  852  864  534  634]  
 [ 606  671  999  665  422  932  864  828  625  566]  
 [ 541  527  722  510  361  661  534  625  354  277]  
 [ 531  505  644  515  320  575  634  566  277  296]]
```


Ex_b:

```
[ [5.27554593e+03 2.11484498e+04 2.18697607e+06 9.04280625e+05
  1.39400192e+05 7.28720782e+04 2.54563046e+04 5.27554593e+03
  2.46632021e+06 1.03072196e+06]
[3.95729622e+00 2.11484498e+04 6.15757328e+04 2.34703341e+01
  6.76773187e+01 3.43857129e+03 9.43987454e+06 5.80567199e-01
  1.17292656e+06 6.85887887e+06]
[3.93366218e-01 8.49590170e+06 2.40397444e+00 1.57365074e+07
  3.50830858e+06 1.17292656e+06]
[1.93651777e+06 4.36142981e+04 3.93366218e-01 1.16167444e+07
  3.23874665e+02 5.27554593e+03 3.05495065e+04 1.44606647e+04
  3.12365659e+06 8.60188419e+04]
[1.42364795e+07 1.75154217e+04 2.77748331e+06 1.18964327e+05
  6.85887887e+06 7.28720782e+04 1.86364462e+00 8.60188419e+04
  1.28666098e+07 1.44606647e+04]
[2.32683564e+07 2.34703341e+01 1.90090548e+05 4.93242050e+06
  5.80567199e-01 3.43857129e+03 2.56940287e+05 5.80567199e-01
  2.77748331e+06 5.07215057e+00]
[5.51196186e+06 6.15224538e+06 2.46632021e+06 1.04774306e+07
  3.43857129e+03 1.01282873e+05 1.08880130e-01 6.15757328e+04
  1.18964327e+05 3.44406121e+05]
[5.26633964e+05 1.71225209e+06 1.08880130e-01 2.40397444e+00
  5.18930312e+01 3.44406121e+05 7.28720782e+04 2.38313319e-01
  7.97838227e+03 1.18964327e+05]
[2.46632021e+06 1.33262411e+06 2.46632021e+06 3.43857129e+03
  4.57991148e+05 1.49372962e+02 3.08726437e+00 1.62968112e+05
  2.21239280e+05 1.90090548e+05]
[2.18697607e+06 3.43857129e+03 3.23874665e+02 1.33262411e+06
  6.04449717e+05 2.34703341e+01 4.40843348e+06 1.08779577e+00
  1.49372962e+02 7.92024810e+05]]
```

```

Ex_c:
[47 71 79 11 47 53 91 73 87 71 95  3  9 73 77 51  3 93 49 45 81 55 95 57
 55 45 85 87 79  1 53 57 67  1  9 21 57 79 79 25 59 61 25]
Ex_d:
[47 71 79 11 47 53 73 71  3 73  3 79 53 67  2 79 79 59 61]
Ex_e:
[[11 47 53 18 22 38 91  4 73 88]
 [79 74 79 38 66 25 10 59 61 60]]
Ex_f:
[[77 51  3 93 28 40 49 45 81 55]]

```

Hình 19. 2nd output

2.3 Lần chạy thứ 3

```

Matrix A:
[[ 74  53  11  24  3  79  26 100  44  17]
 [ 62  17  7  90 80  34  53  2  2  51]
 [ 68  25  8  33 65  25  59  84  34  3]
 [ 80  88 42  56 36  61  43  28  15 91]
 [ 93  80 56  15 47  76  99  1  19 84]
 [ 24  11 40  69 82  38  84  43  10 19]
 [ 12  94 92  3  85  84  69  80  44 22]
 [ 6  76 62 100 68  52  96  57  78 82]
 [ 64  52 75  44 19  36  93  2  35 58]
 [ 8  37 81  57 18  78  89  8  24 14]]
Matrix B:
[[ 4 20 19  5  6  9 15 11  5 17]
 [ 4  9  2 20  5  9 14 19  3 20]]
Matrix C:
[[ 5 19]
 [16 14]
 [19  3]
 [ 3  7]
 [18  1]
 [15  5]
 [16 15]
 [17 18]
 [16 20]
 [ 1 10]]

```

Hình 20. 3rd input

```
Ex_a:
[[ 340  506  300  549  297  399  503  662  334  534]
 [ 506  926  771  661  695  660 1038 1022  676  750]
 [ 300  771  750  301  594  558  812  771  557  506]
 [ 549  661  301  422  214  395  569  739  575  544]
 [ 297  695  594  214  320  444  639  478  327  484]
 [ 399  660  558  395  444  436  742  670  460  551]
 [ 503 1038  812  569  639  742 1038 1144  782  838]
 [ 662 1022  771  739  478  670 1144 1172  775  940]
 [ 334  676  557  575  327  460  782  775  350  789]
 [ 534  750  506  544  484  551  838  940  789  462]]
```

Ex_b:

```
[[1.33262411e+06 6.15757328e+04 3.95729622e+00 1.14884462e+02
 3.93366218e-01 2.46632021e+06 1.93823709e+02 2.32683564e+07
 1.19000372e+04 1.80702704e+01]
[2.56940287e+05 1.80702704e+01 1.43324217e+00 8.49590170e+06
 2.77748331e+06 1.39705727e+03 6.15757328e+04 2.38313319e-01
 2.38313319e-01 4.36142981e+04]
[6.04449717e+05 1.49372962e+02 1.86364462e+00 1.10492833e+03
 3.97543528e+05 1.49372962e+02 1.62968112e+05 4.40843348e+06
 1.39705727e+03 3.93366218e-01]
[2.77748331e+06 6.85887887e+06 7.97838227e+03 1.01282873e+05
 2.20832794e+03 2.21239280e+05 9.76040682e+03 3.23874665e+02
 1.07888516e+01 9.43987454e+06]
[1.16167444e+07 2.77748331e+06 1.01282873e+05 1.07888516e+01
 2.11484498e+04 1.71225209e+06 2.11305399e+07 1.08880130e-01
 3.05412197e+01 4.40843348e+06]
[1.14884462e+02 3.95729622e+00 5.27554593e+03 6.92517140e+05
 3.50830858e+06 3.43857129e+03 4.40843348e+06 9.76040682e+03
 3.08726437e+00 3.05412197e+01]
[5.07215057e+00 1.28666098e+07 1.04774306e+07 3.93366218e-01
 4.93242050e+06 4.40843348e+06 6.92517140e+05 2.77748331e+06
 1.19000372e+04 6.76773187e+01]
[1.08779577e+00 1.71225209e+06 2.56940287e+05 2.32683564e+07
 6.04449717e+05 5.18930299e+04 1.57365074e+07 1.18964327e+05
 2.18697607e+06 3.50830858e+06]
[3.44406121e+05 5.18930299e+04 1.51170871e+06 1.19000372e+04
 3.05412197e+01 2.20832794e+03 1.16167444e+07 2.38313319e-01
 1.75978154e+03 1.39400192e+05]
[1.86364462e+00 2.76078712e+03 3.12365659e+06 1.18964327e+05
 2.34703341e+01 2.18697607e+06 7.63791899e+06 1.86364462e+00
 1.14884462e+02 8.36947167e+00]]
```

```

Ex_c:
[53 11  3 79 17 17  7 53 51 25 33 65 25 59  3 61 43 15 91 93 15 47 99  1
 19 11 69 43 19  3 85 69 57 75 19 93 35 37 81 57 89]
Ex_d:
[53 11  3 79 17 17  7 53  2  2 59  3 61 43 47 19 11 43 19  3 19  2 37 89]
Ex_e:
[[ 74  53  11  24  3  79  26 100  44  17]
 [ 62  17  7  90  80  34  53  2  2  51]]
Ex_f:
[[93 80 56 15 47 76 99  1 19 84]]

```

Hình 21. 3rd output

2.4 Lần chạy thứ 4

```

Matrix A:
[[ 69  46  13  46  50  35  63  19  18  60]
 [ 77  71  23  48  98  71  95  40   6  51]
 [ 20  62  73  50  55  50  98  82  58  68]
 [ 20  83  36  95  45   7  21  58  24  61]
 [ 63  35  99   1  65  16  59 100  66  81]
 [ 77  75  70  24  66  13  91  64  17  25]
 [ 18  98  22  29  29  85  42  50  14  97]
 [ 81  23  35 100  79  58  92  53  72  48]
 [ 59  35  56  15  23  94  51  26  99  48]
 [  5  96  43  48  31  88   6  86  28  45]]

Matrix B:
[[ 7 11  2 14 12  8  1  4 20 20]
 [ 9  1  7 19 15 20 11 15 14 16]]

Matrix C:
[[13  4]
 [ 8 11]
 [12 20]
 [14 16]
 [11 20]
 [19  9]
 [19 20]
 [ 4  1]
 [11  3]
 [ 8 11]]

```

Hình 22. 4th input

Ex_a:

```
[[ 392  425  351  566  586  510  451  249  497  544]
 [ 425  340  330  622  535  648  551  305  479  582]
 [ 351  330  474  774  760  717  530  480  677  764]
 [ 566  622  774 1190  988  900  886  529  754  966]
 [ 586  535  760  988  994  933  847  586  766  913]
 [ 510  648  717  900  933  690  846  385  765  921]
 [ 451  551  530  886  847  846  562  533  769  932]
 [ 249  305  480  529  586  385  533  168  281  427]
 [ 497  479  677  754  766  765  769  281  722  658]
 [ 544  582  764  966  913  921  932  427  658  762]]
```

Ex_b:

```
[ [6.92517140e+05 1.75154217e+04 6.50886125e+00 1.75154217e+04
  3.65542522e+04 1.75978154e+03 2.97779025e+05 3.05412197e+01
  2.34703341e+01 1.90090548e+05]
[1.93651777e+06 9.04280625e+05 8.82218690e+01 2.54563046e+04
 1.91714266e+07 9.04280625e+05 1.42364795e+07 5.27554593e+03
 1.08779577e+00 4.36142981e+04]
[3.97955983e+01 2.56940287e+05 1.17292656e+06 3.65542522e+04
 8.60188419e+04 3.65542522e+04 1.91714266e+07 3.50830858e+06
 1.39400192e+05 6.04449717e+05]
[1.44606647e+04 1.43324217e+00 5.18930312e+01 1.39400192e+05
 1.14884462e+02 2.21239280e+05]
[2.97779025e+05 1.75978154e+03 2.11305399e+07 1.08880130e-01
 3.97543528e+05 1.39447040e+01 1.62968112e+05 2.32683564e+07
 4.57991148e+05 3.12365659e+06]
[1.93651777e+06 1.51170871e+06 7.92024810e+05 1.14884462e+02
 4.57991148e+05 6.50886125e+00 9.43987454e+06 3.44406121e+05
 1.80702704e+01 1.49372962e+02]
[2.34703341e+01 1.91714266e+07 6.76773187e+01 4.16813571e+02
 4.16813571e+02 4.93242050e+06 7.97838227e+03 3.65542522e+04
 8.36947167e+00 1.73775929e+07]
[3.12365659e+06 8.82218690e+01 1.75978154e+03 2.32683564e+07
 2.46632021e+06 1.39400192e+05 1.04774306e+07 6.15757328e+04
 1.03072196e+06 2.54563046e+04]
[1.62968112e+05 1.75978154e+03 1.01282873e+05 1.07888516e+01
 8.82218690e+01 1.28666098e+07 4.36142981e+04 1.93823709e+02
 2.11305399e+07 2.54563046e+04]
[8.08373189e-01 1.57365074e+07 9.76040682e+03 2.54563046e+04
 6.83497590e+02 6.85887887e+06 1.08779577e+00 5.51196186e+06
 3.23874665e+02 1.44606647e+04]]
```



```

Ex_c:
[69 13 35 63 19 77 71 23 71 95 51 73 55 83 95 45  7 21 61 63 35 99  1 65
 59 81 77 75 13 91 17 25 29 29 85 97 81 23 35 79 53 59 35 15 23 51 99  5
 43 31 45]
Ex_d:
[13 19 71 23 71 73 83  7 61 59 13 17 29 29 97 23 79 53 59 23  5 43 31]
Ex_e:
[[ 77  71  23  48  98  71  95  40  6  51]
 [ 20  83  36  95  45  7  21  58  24  61]
 [ 18  98  22  29  29  85  42  50  14  97]
 [ 81  23  35 100  79  58  92  53  72  48]
 [  5  96  43  48  31  88  6  86  28  45]]
Ex_f:
[[77 75 70 24 66 13 91 64 17 25]]

```

Hình 23. 4th output

2.5 Lần chạy thứ 5

```
Matrix A:
[[69 56 69 25 74 37  1 24 27 11]
 [35 72 47 31 17 27 92 21 33 62]
 [45 78 12 20 51 88 60 42 99 19]
 [56 68  6 96 83 57 75 83 30 17]
 [44 23 50 32 21 48 82 86 37 82]
 [92  3 91 88 84  9 76 51 35 48]
 [30 54 89 10 33 18 98 79 45 71]
 [85 38 92 68 73 24  7 78  6  5]
 [23 53 22 98 73 37 74 34 39 85]
 [11 47 55 78 96  1 91 61 71 64]]

Matrix B:
[[ 2  7  9 18 18 20  2  5 17  5]
 [20  9  8 14 15 17 14 12  6 20]]

Matrix C:
[[11  8]
 [ 8  9]
 [19  9]
 [ 4 17]
 [ 4  7]
 [12  3]
 [16 12]
 [12 12]
 [15  3]
 [18 15]]
```

Hình 24. 5th input

Ex_a:

```
[[502 436 495 739 584 569 437 524 375 573]
 [436 418 483 550 410 454 508 399 408 590]
 [495 483 510 666 670 844 553 541 657 631]
 [739 550 666 812 612 772 787 759 610 989]
 [584 410 670 612 396 592 689 659 535 887]
 [569 454 844 772 592 600 684 615 645 784]
 [437 508 553 787 689 684 596 502 535 728]
 [524 399 541 759 659 615 502 564 427 636]
 [375 408 657 610 535 645 535 427 624 687]
 [573 590 631 989 887 784 728 636 687 908]]
```

Ex_b:

```
[ [6.92517140e+05 1.01282873e+05 6.92517140e+05 1.49372962e+02
  1.33262411e+06 2.76078712e+03 1.08880130e-01 1.14884462e+02
  2.50894552e+02 3.95729622e+00]
[1.75978154e+03 1.03072196e+06 2.11484498e+04 6.83497590e+02
  1.80702704e+01 2.50894552e+02 1.04774306e+07 5.18930312e+01
  1.10492833e+03 2.56940287e+05]
[1.44606647e+04 2.18697607e+06 5.07215057e+00 3.97955983e+01
  2.11305399e+07 3.05412197e+01]
[1.01282873e+05 6.04449717e+05 1.08779577e+00 1.57365074e+07
  3.93520561e+06 1.18964327e+05 1.51170871e+06 3.93520561e+06
  5.34672010e+02 1.80702704e+01]
[1.19000372e+04 8.82218690e+01 3.65542522e+04 8.70628432e+02
  5.18930312e+01 2.54563046e+04 3.50830858e+06 5.51196186e+06
  2.76078712e+03 3.50830858e+06]
[1.04774306e+07 3.93366218e-01 9.43987454e+06 6.85887887e+06
  4.40843348e+06 2.40397444e+00 1.71225209e+06 4.36142981e+04
  1.75978154e+03 2.54563046e+04]
[5.34672010e+02 7.28720782e+04 7.63791899e+06 3.08726437e+00
  1.10492833e+03 2.34703341e+01 1.91714266e+07 2.46632021e+06
  1.44606647e+04 9.04280625e+05]
[4.93242050e+06 3.43857129e+03 1.04774306e+07 6.04449717e+05
  1.17292656e+06 1.14884462e+02 1.43324217e+00 2.18697607e+06
  1.08779577e+00 8.08373189e-01]
[8.82218690e+01 6.15757328e+04 6.76773187e+01 1.91714266e+07
  1.17292656e+06 2.76078712e+03 1.33262411e+06 1.39705727e+03
  4.26693226e+03 4.93242050e+06]
[3.95729622e+00 2.11484498e+04 8.60188419e+04 2.18697607e+06
  1.57365074e+07 1.08880130e-01 9.43987454e+06 2.21239280e+05
  9.04280625e+05 3.44406121e+05]]
```

```

Ex_c:
[69 69 25 37  1 27 11 35 47 31 17 27 21 33 45 51 99 19 83 57 75 83 17 23
 21 37  3 91  9 51 35 89 33 79 45 71 85 73  7  5 23 53 73 37 39 85 11 47
 55  1 91 61 71]
Ex_d:
[37 11 47 31 17 19 83 83 17 23 37  3 89 79 71 73  7  5 23 53 73 37 11 47
 61 71]
Ex_e:
[[23 53 22 98 73 37 74 34 39 85]
 [11 47 55 78 96  1 91 61 71 64]]
Ex_f:
[[11 47 55 78 96  1 91 61 71 64]]

```

Hình 25. 5th output

TÀI LIỆU THAM KHẢO

Tiếng Anh

1. Practice Labs. 2024. Applied Linear Algebra for IT. Faculty of Information Technology. Ton Duc Thang University.
2. Practical Data Science with python, “Vectorization in python” 2024. [Online]. Address:
https://www.practicaldatascience.org/notebooks/class_2/week_4/11_vectorization.html